USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

The Launcher, and Four Defects It Found by Existing

UM-NATOS-021

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-021	1.0 · 2026-08-15	**Complete, verified on hardware**	Internal

1. Abstract

The first thing in this project that exists for a user rather than for a test. Everything before it was verified by reading serial output; this is verified by someone touching the glass and a program starting.

It is about 250 lines. Its value turned out to be less in what it does than in what it **demanded** — it is the first consumer to care where across the screen a finger is, the first to run the display at a rate that mattered, and the first to need a task slot. Each of those exposed a defect that had been latent for weeks:

exposed	defect	recorded in
horizontal position	touch X axis inverted since the driver was written	UM-NATOS-017 §7.1
position validity	PENIRQ answers presence, not validity; rail samples	UM-NATOS-017 §4.1
a task slot	nine `task_create` calls against a `TASK_MAX` of 8 — no idle task	§5 below
display throughput	applications and renderer blocking each other on the draw lock	UM-NATOS-014 §10

None was found by inspection. All four were found by building something that depended on them.

2. A launcher, not a window manager

Applications still render into the fixed horizontal strips `app.c` assigns by slot index. There is no focus model, no z-order, and no way for a program to own the screen and hand it back.

That boundary is deliberate rather than unfinished. Dynamic viewports need an arbitration policy for who owns which pixels, and building one before a user interface exists to demand it is designing against a guess — the same reason focus arbitration was deferred while viewports could not overlap. A launcher is the consumer that makes the question concrete; it is not yet the consumer that answers it.

2.1 Layout

```
240 x 168 region, 3 x 3 grid over 154 rows, 14-row status strip

  +----------+----------+----------+
  | counter  | squares  | draw     |   cell 80 x 51
  +----------+----------+----------+
  | paint    | blit     | ping     |
  +----------+----------+----------+
  | pong     | rogue    | 3D view  |
  +----------+----------+----------+
  | started counter                |   status strip, expires after ~2 s
  +--------------------------------+
```

The 3D view is an icon rather than a permanent fixture: it becomes something you open, returning via the top-left corner.

2.2 Launch by name, never by index

`shell_launch(name)` looks the program up in the shell's table. The launcher holds no image pointers of its own.

This is not fastidiousness. An earlier launcher in this project indexed the table directly and silently started a *different* program than the one it named, because the table had been reordered — a `rogue` icon running `gfxrogue`. A name lookup cannot drift out of step with the table it reads.

2.3 The launcher costs nothing when idle

It repaints only when something changed, so an idle desktop pushes **zero** pixels — unlike the raycaster, which repaints unconditionally because every frame differs. If the screen looks frozen, that is correct: it is waiting for input.

3. Why a cursor on a touchscreen

A cursor is an indirection — you can already point at what you want. It earns its place here for two measured reasons: the panel is resistive and noisy, and an icon large enough to hit reliably by direct tap would leave room for very few.

The interaction is **hybrid, not modal**: a single tap MOVES the cursor to it, a double-tap OPENS what it is on. A confident user taps twice and it opens; an unsure one taps, sees where the cursor landed, and corrects. Neither has to be told which mode they are in.

4. Two defects of its own

4.1 Status text drawn over a control

The launch message was drawn at `DESK_H - 10`, which is inside the bottom-left cell's label. So "started" appeared over `pong`'s name — and `g_msg_sel` was never cleared, so it stayed there permanently.

It was reported as "*it selected Pong, which was a bit off*", and that reading was entirely reasonable. The interface was asserting something false about its own state, indefinitely.

Text that overlaps a control is not a cosmetic problem. The fix gives status its own strip below the grid, shrinks the cells to make room, excludes the strip from hit-testing, names what launched rather than saying "started", and expires after ~2 s — because a status line that never clears stops describing the present and becomes decoration.

4.2 The selection was read at the worst possible moment

Selection followed *every* sample while the finger was down, so the **last** sample before release decided it — and release is precisely when a resistive panel produces its worst data. Measured, one tap on the top-right icon:

```
first sample -> cell 2      (correct)
last sample  -> cell 3      (wrong)
```

The correct target was read, recorded, and discarded microseconds later.

Selection is now latched from the **first** sample of a press and not moved again until the finger lifts. Later samples describe a finger being *lifted*, which is not an instruction. Drag no longer moves the cursor; tap again to correct.

This was diagnosed by an instrument built for the theory and then nearly deleted when the theory looked wrong. The first/last cell pair is still reported, with a note saying why: removing an instrument once it has found its bug is how the bug returns unnoticed.

5. There was no idle task

`kmain` makes **nine** `task_create` calls; `TASK_MAX` was **8**. The ninth is the idle task, so it failed: `task_create()` returns −1 when the table is full, `task_set_idle(-1)` bounds-checks and quietly does nothing, and the return value was never printed or checked.

Two real costs. `WAITI` never executed, so the CPU never slept. And with every task asleep the scheduler found nothing runnable and fell through to "resume the interrupted context" — which resumes a **sleeping** task, the exact invariant blocking exists to enforce.

The evidence had been printed and read. The `stacks` table added hours earlier listed ids 0–7 with no idle task; it was looked at and not registered. That is the failure UM-NATOS-019 is entirely about, committed while writing the report about it.

`TASK_MAX` is now 12 and `must_create()` panics rather than returning −1, so a full table cannot fail quietly again.

6. Verification

```
tap left icon   raw_x=3408 -> x=0,   cell 0
tap right icon  raw_x=920  -> x=196, cell 2
seven consecutive taps, z 1662..2261, no rail samples
x spanning 0..187 across all three columns
first and last sample of each press agree
```

All three columns selectable, double-tap opens the named program, and the status strip reports which. Confirmed on hardware by the user.

7. Metrics

Quantity	Value
Grid	3 × 3, cells 80 × 51 px
Status strip	14 px, message expires ~2 s
Double-tap window	60 ticks (~600 ms)
Minimum press	2 ticks, rejects contact chatter
SPI cost when idle	0 bytes
Defects exposed in other subsystems	4
Defects of its own	2

8. What this does not establish

- **No focus, no windows, no z-order.** §2. Applications occupy fixed strips and cannot be brought forward.
- **No way to stop a program from the launcher.** `kill` exists only in the shell; the icon grid can start and cannot stop.
- **The running-program marker is per-name, not per-instance.** It now compares the whole name (it originally compared the first character, which marked `paint`, `ping` and `pong` together — found while writing this section, not by looking at the screen, because three wrong four-pixel dots read as a rendering artefact). Two instances of the same program would still share one marker.
- **Double-tap timing is unmeasured.** 600 ms and the 2-tick minimum press were reasoned, not derived from anyone's actual tapping. The counters exist to settle them and nobody has.
- **No icons.** Coloured rectangles with labels, because there is still no asset pipeline (UM-NATOS-011 §6).
- **One user.** Every judgement about whether the interaction feels right came from a single person on a single panel.

9. References

- UM-NATOS-017 §4.1, §7.1 — the touch defects this exposed
- UM-NATOS-014 §10 — the lock contention this exposed
- UM-NATOS-019 — verifying that a mechanism exists is not verifying it works
- UM-NATOS-016 §2 — the fixed viewport model the launcher does not change
- `kernel/desktop.c` — grid, cursor, press latching, status strip
- `kernel/shell.c` — `shell_launch()`, the single lookup path